

Experimental Full-Pair QK Relational Architecture

Headless distance-aware reading with separate strict multi-hop propagation

Contact: rickeyernest1990@gmail.com

Publication date: 14 September 2026

Contents

Overview	2
1 Motivation and scope	2
2 Architecture	2
2.1 Distance-aware ordinary QK reading	2
2.2 Separate strict hop field	3
2.3 ASCII architecture sketch	3
3 Python implementation	4
3.1 Tested small-model configuration	4
3.2 Core PyTorch operator	4
3.3 Two-layer language model	6
3.4 Getting it running	7
3.5 Implementation verification	7
4 Experimental protocol	8
5 Matched real-text competitor test	8
5.1 Held-out language result	8
5.2 Parameter use and arithmetic accounting	9
6 Answer-only multi-hop relationship test	9
6.1 Why the strict hop mask matters	9
6.2 Held-out depth behaviour	9
6.3 Three-seed transition diagnostic	10

7	Runtime and memory measurements	10
8	What the experiments currently support	11
9	Limits of the current evidence	12
10	Publication, contact and licensing	12
11	Conclusion	13

Licensing warning - noncommercial use. This publication uses a split licence. Documentation, explanatory text, figures and tables are licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). Source code and code listings implementing the experimental architecture are licensed separately under the PolyForm Noncommercial License 1.0.0.

Overview

This note describes an experimental language-model architecture based on full token-to-token query-key relationships. The ordinary path is headless and has no learned value projection: it uses one distance-aware QK field to read raw token state. In the final block, an optional relational path uses separate learned hop projections, Q_h and K_h , with a strictly causal no-self mask. Requested extra depth is then produced by repeated application of the same learned hop matrix rather than by adding network layers.

The evidence reported here is deliberately split into two questions. First, can the architecture remain competitive with a genuine four-head Transformer on ordinary language modelling at equal total parameter count? Second, can the explicit hop operator learn reusable multi-step composition using only final-answer supervision? The current tests answer both questions positively at small scale, but they do not establish general superiority. The model remains a full-pair $O(N^2)$ architecture, the current dense implementation has no measured memory or inference-speed advantage, and the synthetic hop task is not a substitute for natural-language reasoning evaluation.

1 Motivation and scope

The current research question is:

Can a two-layer, headless full-pair QK language model match a parameter-matched four-head Transformer while retaining a separately learned, explicitly controllable relational operator whose depth can be increased at inference?

The aim is not to claim a foundation-model replacement. The aim is to test a specific trade: ordinary language processing should remain competitive at fixed size, while optional extra relational computation is invoked through A_h^k without adding layers or changing the stored parameter count.

2 Architecture

2.1 Distance-aware ordinary QK reading

Let the normalized token-state matrix in a block be $H \in \mathbb{R}^{N \times d}$. Ordinary reading computes

$$Q_0 = HW_{Q_0}, \quad K_0 = HW_{K_0}, \quad (1)$$

and

$$C_0 = \frac{Q_0 K_0^T}{\sqrt{d}}. \quad (2)$$

There is no learned V projection in the QK operator. The transmitted content is the token state itself, and there are no attention heads.

The fixed long-tailed ordinary distance prior is

$$D(r) = \frac{0.02}{1+r} + \frac{0.98}{(1+r)^2}. \quad (3)$$

For causal language use, define $M_{\leq}$ to allow only $j \leq i$. The ordinary matrix is

$$A_0 = \text{softmax} \left(\frac{Q_0 K_0^T}{\sqrt{d}} + \log D(|i-j|) + M_{\leq} \right). \quad (4)$$

Normal operation uses $A_0 H$.

2.2 Separate strict hop field

The hop pathway is intentionally given a different learned similarity geometry:

$$Q_h = HW_{Q_h}, \quad K_h = HW_{K_h}. \quad (5)$$

Its mask is strictly causal. Define $M_{<}$ to allow only $j < i$, so an explicitly requested hop cannot remain on the same relation. The implementation gives row zero a harmless fallback self-entry to avoid an empty softmax row. The hop matrix is

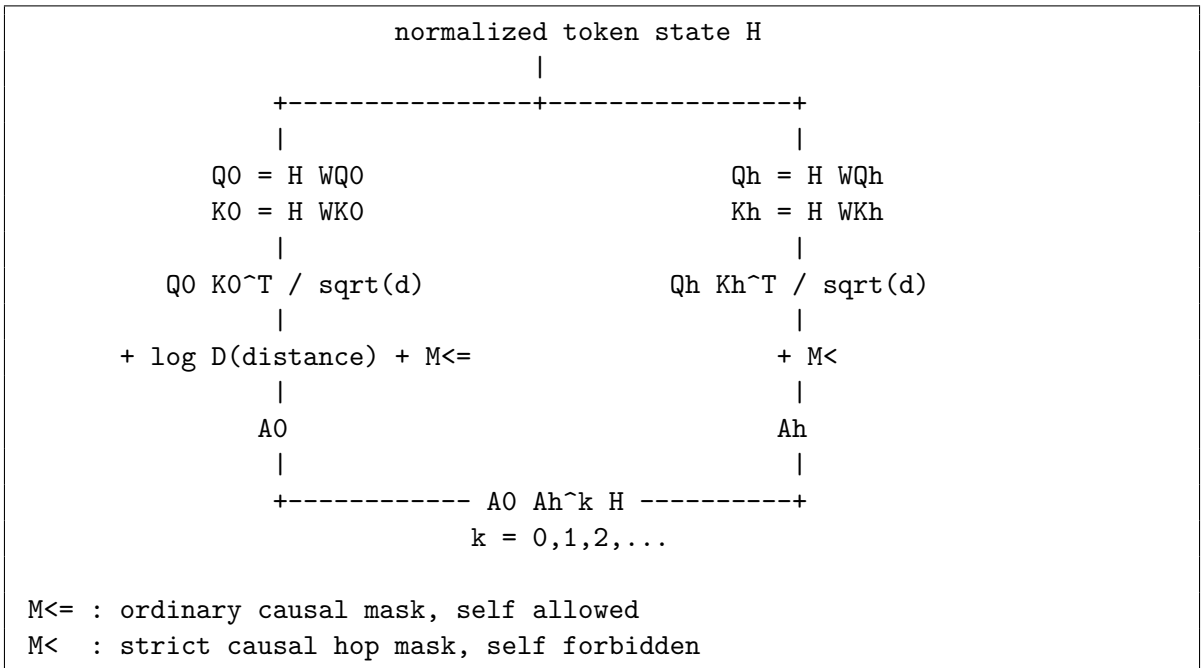
$$A_h = \text{softmax} \left(\frac{Q_h K_h^T}{\sqrt{d}} + M_{<} \right). \quad (6)$$

Distance degradation is not reapplied inside the hop field. For k requested extra hops, the block output is

$$Z_k = A_0 A_h^k H. \quad (7)$$

The separate Q_h, K_h projections are used only in the final QK block. At $k = 0$ they are dormant.

2.3 ASCII architecture sketch



3 Python implementation

3.1 Tested small-model configuration

The tested language model can be implemented directly in PyTorch without custom kernels. The configuration used for the matched language comparison is two QK blocks, width $d = 48$, byte vocabulary 256, fixed sinusoidal positions, feed-forward width 240 in the first block and 192 in the final block, and separate Q_h, K_h projections only in the final block. This gives exactly 80,352 trainable parameters.

Normal language operation calls the model with `extra_hops=0`. In that mode the final Q_h, K_h projections are not evaluated. When `extra_hops=k` is requested, the final block computes A_h once, applies the same matrix k times to the normalized state, and then performs the ordinary A_0 read. This is exactly $A_0 A_h^k H$; it does not recompute Q_h, K_h on every hop.

3.2 Core PyTorch operator

The essential implementation is below. The ordinary mask allows self-attention, while the hop mask forbids self-transitions except for row zero, which has no legal earlier key and therefore receives a single fallback self-entry to keep softmax finite.

```
def distance_prior_log(n: int, device, dtype):
    """Return delta=i-j and log D(|i-j|), with D(r)=.02/(1+r)+.98/(1+r)^2."""
    pos = torch.arange(n, device=device)
    delta = pos[:, None] - pos[None, :]
    r = delta.abs().to(dtype)
    log_prior = torch.log(0.02 / (1.0 + r) + 0.98 / (1.0 + r).square())
    return delta, log_prior

def qk_fields(h, q0, k0, qh=None, kh=None, need_hop=False):
    """Build ordinary A0 and, optionally, strict hop Ah.

    Shapes:
        h, q0, k0, qh, kh: [batch, sequence, d_model]
        A0, Ah:           [batch, sequence, sequence]
    """
    n = h.shape[1]
    d = q0.shape[-1]
    delta, log_prior = distance_prior_log(n, h.device, h.dtype)

    # Ordinary causal reading: j <= i, so the diagonal/self is legal.
    ordinary_mask = delta >= 0
    scores0 = (q0 @ k0.transpose(-2, -1)) / math.sqrt(d)
    scores0 = scores0 + log_prior
    A0 = scores0.masked_fill(~ordinary_mask, -torch.inf).softmax(dim=-1)

    Ah = None
```

```

if need_hop:
    if qh is None or kh is None:
        raise ValueError("qh and kh are required when need_hop=True")

    # Explicit hop:  $j < i$ . A hop must move to an earlier relation.
    strict_mask = delta > 0
    strict_mask = strict_mask.clone()

    # Row 0 has no earlier key. A single fallback self entry keeps softmax finite.
    # No later row is allowed to self-loop.
    strict_mask[0, 0] = True

    scoresh = (qh @ kh.transpose(-2, -1)) / math.sqrt(d)
    Ah = scoresh.masked_fill(~strict_mask, -torch.inf).softmax(dim=-1)

return A0, Ah

```

```

class QKBlock(nn.Module):
    def __init__(self, d_model: int, ff_dim: int, has_hop: bool):
        super().__init__()
        self.has_hop = has_hop
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)

        self.q0 = nn.Linear(d_model, d_model, bias=False)
        self.k0 = nn.Linear(d_model, d_model, bias=False)

        if has_hop:
            self.qh = nn.Linear(d_model, d_model, bias=False)
            self.kh = nn.Linear(d_model, d_model, bias=False)

        self.ff = nn.Sequential(
            nn.Linear(d_model, ff_dim, bias=False),
            nn.ReLU(),
            nn.Linear(ff_dim, d_model, bias=False),
        )

    def forward(self, x, extra_hops: int = 0):
        h = self.ln1(x)
        q0, k0 = self.q0(h), self.k0(h)

        if extra_hops > 0:
            if not self.has_hop:
                raise ValueError("extra_hops may only be applied to the hop-enabled final block")
            qh, kh = self.qh(h), self.kh(h)
            A0, Ah = qk_fields(h, q0, k0, qh, kh, need_hop=True)

            #  $A0 \leftarrow Ah \tilde{k} H$ : apply Ah repeatedly to H, then make the ordinary A0

```

```

read.
    z = h
    for _ in range(extra_hops):
        z = Ah @ z
    out = AO @ z
else:
    AO, _ = qk_fields(h, q0, k0, need_hop=False)
    out = AO @ h

x = x + out
return x + self.ff(self.ln2(x))

```

3.3 Two-layer language model

The tested parameter matching depends on the feed-forward widths shown below. The first block has no hop projections; the second block contains Q_h, K_h and uses the narrower feed-forward layer so the complete QK model remains exactly 80,352 parameters.

```

class QKLanguageModel(nn.Module):
    def __init__(self, vocab_size=256, d_model=48, max_context=2048):
        super().__init__()
        self.vocab_size = vocab_size
        self.d_model = d_model

        self.embed = nn.Embedding(vocab_size, d_model)
        self.blocks = nn.ModuleList([
            QKBlock(d_model, ff_dim=240, has_hop=False),
            QKBlock(d_model, ff_dim=192, has_hop=True),
        ])
        self.norm = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)

        # Fixed sinusoidal positions, matching the tested small language model
        .
        pos = torch.arange(max_context)[: , None].float()
        div = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
d_model)
        )
        pe = torch.zeros(max_context, d_model)
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("position", pe, persistent=False)

        self.apply(self._init_weights)

    @staticmethod
    def _init_weights(module):
        if isinstance(module, (nn.Linear, nn.Embedding)):

```

```

nn.init.normal_(module.weight, std=0.02)

def forward(self, tokens, extra_hops: int = 0):
    if tokens.shape[1] > self.position.shape[0]:
        raise ValueError("sequence is longer than max_context")

    h = self.embed(tokens) * math.sqrt(self.d_model)
    h = h + self.position[: tokens.shape[1]]

    h = self.blocks[0](h, extra_hops=0)
    h = self.blocks[1](h, extra_hops=extra_hops)
    return self.head(self.norm(h))

def parameter_count(model):
    return sum(p.numel() for p in model.parameters())

```

3.4 Getting it running

Install PyTorch and NumPy, save the companion implementation as `qk_minimal.py`, and first run the built-in architecture checks:

```

python -m pip install torch numpy
python qk_minimal.py --smoke

```

The smoke test checks the exact 80,352 parameter count, finite normal and hopped forward passes, row-normalized A_0 and A_h , the strict no-self hop diagonal, and that Q_h, K_h are dormant at $k = 0$ but receive finite gradients when a hop is requested.

A minimal next-byte training run on any local byte file can then be started with:

```

python qk_minimal.py --train-byte-file version9.txt \
    --steps 4096 --context 128 --batch 8 --seed 11

```

The compact training loop uses AdamW at 5×10^{-4} , weight decay 0.01 and gradient clipping at 1.0, matching the main small-model training settings. It is intended to make the architecture immediately runnable; reproducing the exact report tables additionally requires the fixed train/test splits, checkpoints and evaluation windows from the experiment package.

3.5 Implementation verification

The standalone companion implementation was checked against the saved benchmark implementation used for the competitor audit. After loading identical weights, it produced bit-identical logits on the same random inputs for `extra_hops=0, 1, 3` and `8`, and both implementations contain exactly 80,352 parameters. A two-step byte-training smoke run

also completed with finite loss and gradients. These checks verify that the code listing implements the mathematics reported in this document rather than an approximate rewrite.

4 Experimental protocol

The experiments reported here were run in bounded, checkpointed batches so completed work survived execution or message timeouts. Negative and mixed results were retained rather than tuned away.

The primary language comparison used the Vim 9.1 `version9.txt` byte stream. The first 1,450,000 bytes were used for training, the next 80,000 for validation, and the remaining 80,934 for test. The comparison used byte vocabulary 256, context 128, batch 8, 4,096 optimizer updates, AdamW at 5×10^{-4} , and seeds 11, 22 and 33. Each model/seed therefore saw 4,194,304 next-byte target presentations. Across both architectures and all three seeds, the primary comparison used 25,165,824 training target presentations. Evaluation scored the same 32,768 fixed held-out targets for each model and seed.

Both language models used two layers and width $d = 48$. The Transformer used four heads of 12 dimensions each and feed-forward width 192. The QK model used feed-forward width 240 in its first block and 192 in its final block, where the separate Q_h, K_h projections are located. This compensates for the hop projections so both complete models have exactly 80,352 trainable parameters.

The hop mechanism was tested separately on randomized causal relation chains. That test used 33 chain edges plus 15 distractor edges, fresh graph mappings, no edge-transition labels, no hop-depth feature in the prompt, and only final-answer cross-entropy. Training used 500 normal-only steps followed by 1,000 steps mixing required relation depths 1 through 8. Chance accuracy was $1/34 = 2.94\%$.

5 Matched real-text competitor test

5.1 Held-out language result

Seed	QK loss	Transformer loss	QK accuracy	Transformer accuracy
11	1.7324	1.7510	52.51%	51.75%
22	1.7789	1.7476	52.39%	51.76%
33	1.7434	1.7800	52.74%	50.50%
Mean	1.7516	1.7595	52.55%	51.34%

The mean QK-minus-Transformer loss gap is -0.0080 nats/byte. QK has lower held-out loss on seeds 11 and 33; the Transformer has lower loss on seed 22. QK has higher next-byte accuracy on all three seeds, with a mean gap of $+1.21$ percentage points. Mean $\exp(\text{loss})$ is 5.764 for QK and 5.810 for the Transformer.

With only three seeds, the paired 95% t -interval for the loss difference is $[-0.0955, +0.0795]$, which crosses zero. The result therefore supports competitiveness at this scale, not statistical proof that QK is superior.

5.2 Parameter use and arithmetic accounting

Both models store exactly 80,352 parameters. During ordinary QK operation ($k = 0$), the final Q_h, K_h projections are dormant. Thus 75,744 QK parameters are active and 4,608 parameters, or 5.73% of the model, are reserved for explicit hopping. The Transformer uses all 80,352 stored parameters in its ordinary forward pass.

A simple multiply-accumulate accounting over attention projections, pair products/reads and feed-forward layers, excluding costs common to both models, gives:

Context / batch	QK MACs	Transformer MACs	QK difference
128 / 8	77.07M	81.79M	-5.77%
512 / 8	610.27M	629.15M	-3.00%

Both architectures remain quadratic in sequence length. Explicit hops add compute without adding parameters. At context 128, the approximate QK cost relative to one Transformer forward is 1.15x for +1 hop, 1.38x for +4, 1.69x for +8, 2.31x for +16, and 3.54x for +32.

6 Answer-only multi-hop relationship test

6.1 Why the strict hop mask matters

Earlier variants allowed A_h to attend to its own row position. Diagnostic measurements found large self-loop attraction: shared Q, K placed about 42.8% mean hop mass on self, and separate Q_h, K_h with self still allowed placed about 28.4%. Separate projections alone did not solve deep hopping, and a strict no-self mask with shared projections also did not solve it. The combination of separate Q_h, K_h and strict hopping did.

No transition labels were supplied in this test. Requested depth entered only through the external `extra_hops=k` operation.

6.2 Held-out depth behaviour

For seed 11, evaluated on 1,024 fresh graphs per depth:

Relations required	Extra hops	Dynamic hop QK	Same weights, hops off
1	0	75.0%	75.0%
2	1	76.1%	9.5%
3	2	74.3%	3.2%
4	3	77.2%	1.2%
5	4	74.6%	1.0%
8	7	76.1%	0.1%
12	11	73.7%	0.3%
16	15	75.0%	0.1%
24	23	73.9%	0.0%
33	32	0.7%	0.0%

Depths above 8 were never used during training. The identical-weight no-hop ablation collapses at deeper depths, while dynamic hopping remains high through depth 24. This is direct evidence that A_h^k is contributing useful composition rather than the ordinary two-layer path solving the deeper task by itself.

A finer probe stayed roughly 60–75% through relation depth 31 and rose to about 88% at depth 32 before collapsing at depth 33. The terminal failure was traced to endpoint/readout representation rather than loss of the transition chain.

6.3 Three-seed transition diagnostic

Across 16,384 fresh valid edge-to-next-edge transitions per seed:

Seed	A_h correct-next-edge top-1	Mean mass on correct transition
11	99.994%	99.685%
22	99.982%	99.462%
33	99.994%	99.652%

All three seeds learned the raw transition graph extremely accurately from final-answer supervision alone. End-answer convergence was seed-sensitive: seeds 11 and 33 reached roughly 70–77% on many deep probes, while seed 22 learned the transition graph equally cleanly but had weaker end-answer accuracy within the same fixed 1,500-step budget.

The depth-33 boundary is not evidence that A_h diffuses before reaching the end. Direct diagnostics show the transition relation remains accurate, but the terminal relation is structurally different because it has no successor and its hidden-state readout is poor. The current +32 case is therefore not counted as a solved end-to-end result.

7 Runtime and memory measurements

These measurements were taken on PyTorch 2.10 CPU and should not be generalized to optimized GPU implementations. The Transformer uses PyTorch fused scaled-dot-product attention, while QK is a direct dense research implementation.

Workload	QK	Transformer	Observation
Inference, c128/b8	263k tok/s	435k tok/s	Transformer faster
Train step, c128/b8	84.1k tok/s	74.8k tok/s	QK faster
Inference, c512/b8	128.9k tok/s	184.8k tok/s	Transformer faster
Train step, c512/b8	78.7k tok/s	67.9k tok/s	QK faster

The complete 4,096-step saved training runs took a mean 48.45 seconds for QK and 65.59 seconds for the Transformer on this CPU setup. This agrees with the isolated training-step direction, but it is not a general hardware claim.

Peak process RSS did not show a QK memory advantage:

Workload	QK peak RSS	Transformer peak RSS
Inference, c128/b8	267 MB	264 MB
Training, c128/b8	337 MB	335 MB
Inference, c512/b8	303 MB	276 MB
Training, c512/b8	424 MB	381 MB

Therefore the current implementation supports no memory-efficiency claim. A different fused or blockwise implementation might change constants, but that has not been demonstrated.

8 What the experiments currently support

The evidence supports the following limited statements:

- A two-layer, width-48, headless QK model with no learned value projection can match a same-size four-head Transformer closely on this small byte-level real-text experiment.
- At exactly 80,352 stored parameters, the QK model has slightly better mean held-out loss and accuracy across the three tested seeds, but the loss difference is not statistically established with only three seeds.
- The separate Q_h, K_h plus strict no-self hop field can learn a highly accurate reusable transition operator from final-answer loss alone.
- On the controlled relation task, a model trained only through depth 8 can retain useful answer accuracy far beyond that depth, while the same weights with hops disabled collapse.
- The architecture exposes variable relational compute through A_h^k without changing stored parameter count or adding network layers.
- On this CPU implementation, QK training was faster and simple ordinary-operation MAC accounting was slightly lower than the matched Transformer.

9 Limits of the current evidence

The experiments do not establish that this architecture is a Transformer replacement. Important limits are:

- The primary language corpus is Vim documentation rather than a large standard benchmark corpus.
- Three language seeds are descriptive evidence, not enough to establish statistical superiority.
- The relationship fields remain full-pair and $O(N^2)$ in sequence length.
- The present dense QK implementation is slower at ordinary inference and uses more memory at longer tested context than PyTorch’s fused Transformer baseline.
- Both models behaved poorly and inconsistently when evaluated beyond the context length used for language training; no robust long-context advantage is established.
- The hop task is a controlled synthetic relation problem, not general natural-language reasoning.
- The absolute +32 / depth-33 end-to-end hop case still has a terminal/readout boundary failure.
- A parameter-matched four-head Transformer attempted on the synthetic relation encoding failed to learn even the base one-step task under the fixed schedule. That result is therefore rejected as evidence that QK beats Transformers at reasoning; a stronger Transformer relational control is still required.
- No optimized GPU kernel, large-model scaling study, independent reproduction, or peer review is included.

10 Publication, contact and licensing

Publication date: 14 September 2026

Contact: rickeyernest1990@gmail.com

Licensing. Documentation, explanatory text, figures and tables are licensed under the Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0), subject to attribution and the noncommercial-use restriction.

Source code implementing the experimental architecture, including implementation code and source-code listings reproduced with the work, is licensed separately under the PolyForm Noncommercial License 1.0.0.

Neither licence grants unrestricted commercial-use rights. For commercial licensing enquiries, use the contact above.

These notices apply to copyrightable expression, documentation, presentation and software to the extent protected by applicable law. They are not intended to restrict mathematical

facts, formulas, abstract ideas or other material not subject to copyright protection, and they do not establish or imply patentability, novelty or ownership of ideas governed by separate law or prior art.

11 Conclusion

The architecture is a full-pair quadratic QK attention variant with two distinct learned geometries: distance-aware ordinary reading through A_0 , and separate strict relational propagation through A_h . At equal total parameter count it closely matches a genuine four-head Transformer on a multi-million-target real-text training comparison, while the optional hop field independently learns a reusable compositional transition operator from final-answer supervision alone.

The strongest defensible interpretation is not that QK has beaten the Transformer. The language advantage is small and not statistically established, ordinary inference is currently slower, measured memory is not better, and the synthetic relation test does not prove general reasoning. The distinctive result is that a fixed two-layer headless model can retain Transformer-competitive ordinary language performance while exposing an explicit variable-depth relational operation, $A_0 A_h^k H$, whose learned transition structure extrapolates well beyond the hop depths used in training.

The next decisive tests are larger standard-corpus language runs, more seeds and model scales, an optimized GPU implementation, a stronger Transformer control that demonstrably learns the base relational task, and a repair of the terminal readout boundary. Until those are completed, the architecture is best described as a viable small attention competitor with an additional learned variable-depth relational operator, not a demonstrated replacement for conventional Transformers.